

ACESSÍLIA

Relatório de processo

Adaptação do material “Fórmula de Bhaskara” e os problemas de semântica matemática enfrentados no caminho

Baseado em execução real do pipeline

1. Do que trata este documento

Este relatório explica, com base numa execução real do pipeline, como o ACESSÍLIA transforma um PDF de material didático em versões acessíveis, e conta a história dos dois problemas de semântica matemática que mais deram trabalho: fórmula sendo tratada como texto solto, e texto sendo tratado como se fosse fórmula.

O material de entrada foi um PDF de uma página com a Fórmula de Bhaskara: um parágrafo introdutório, três fórmulas (a forma geral, o discriminante e a fórmula em si) e uma tabela explicando cada componente. É um bom caso de teste porque tem os três tipos de conteúdo que mais geram confusão: texto puro, fórmula isolada e fórmula misturada dentro de frase.

1.1 O log usado como referência

O log a seguir é de uma execução real, do arquivo enviado pelo Telegram até a entrega dos cinco formatos de saída (TXT, HTML, DOCX, PDF e MP3). Ele aparece comentado ao longo do documento — cada trecho relevante é reproduzido e explicado na hora.

2. O processamento, passo a passo

Reproduzo abaixo os trechos principais do log, na ordem em que aparecem, com o que cada um significa.

2.1 Extração da página

```
core.agents.agente_unico:executar:402 - Processando 1
pagina(s) para
formula_de_bhaskara_material_revisado (1)-1.pdf com
estruturador=DoclingStructurer
```

O que está acontecendo: o Docling leu a página e devolveu as regiões brutas; cada bloco de conteúdo com sua posição (bounding box) na página.

2.2 Planejamento: o que é cada região

```
core.agents.agente_unico:_catalogar:608 - [pag 1]
Geometria catalogada:
11 regioao(oes), 11 com spans
core.agents.planejador:planejar:472 - [pag 1] Plano:
5 text_clean, 3 titulo, 3 formula, 1 table
```

O que está acontecendo: o planejador classificou as 11 regiões extraídas: 5 de texto limpo, 3 títulos, 3 fórmulas e 1 tabela. É essa classificação que decide o caminho de cada região... aí o texto vai direto para a saída, fórmula e tabela vão para o especialista de visão computacional. As 3 fórmulas detectadas aqui são exatamente a forma geral, o discriminante e a fórmula de Bhaskara em si.

2.3 Visão computacional em cada fórmula e na tabela

```
core.agents.agente_unico:_process_region_with_vision:1203 -
[pag 1] especialista Agno + critico: formula
(repete 3 vezes - uma por fórmula)
core.agents.critico_visual:descrever_regiao_verificada_com_meta:666
Critico aprovou regioao formula (confianca=0.98)
```

O que está acontecendo: cada fórmula passa por um especialista (que descreve o que vê na imagem) e depois por um crítico visual, que CONFERE a descrição contra a imagem original antes de aceitar. As três fórmulas e a tabela desta página foram aprovadas com confiança 0.98. (estudando um pouco de machine learning gostei muito dessa ideia de ‘tenho tantos por cento de certeza’)

2.4 O aviso sobre metadados ACESSÍLIA

```
core.agents.agente_unico:_registrar_acessilia:1466 -  
Metadado ACESSILIA  
nao registrado (No module named  
'core.services.acessilia_meta')
```

O que está acontecendo: este aviso aparece quatro vezes e merece destaque à parte; está detalhado na seção 6.

2.5 Poda e montagem

```
pipeline.matematica.podador:_podar:577 - Poda: 1 ruído,  
0 duplicata(s), 0 par(es) julgado(s) (14 -> 13)
```

O que está acontecendo: o podador remove ruído do documento montado: aqui, uma região que não carregava conteúdo real (provavelmente um fragmento de geometria sem texto útil) foi descartada, e o documento passou de 14 para 13 blocos. O “0 duplicata(s)” é o resultado da checagem por semelhança estrutural (tem mais sobre na seção 4.4): nenhuma fórmula foi confundida com outra.

2.6 Revisão global e exportação

```
core.agents.editor_textual:revisar_documento:309 - Editor devolveu  
resposta sem JSON reconhecível; marcando necessidade de revisao humana  
core.services.coordenador_de_exportacao:gerar_artefatos:133 -  
Artefatos gerados: docx, html, pdf, txt
```

O que está acontecendo: o editor textual é uma passada final sobre o documento inteiro, procurando inconsistências entre páginas (um termo escrito de duas formas, por exemplo). Nesta execução ele não conseguiu interpretar a resposta do modelo de linguagem como JSON válido — um problema de infraestrutura de IA, não de

conteúdo, por segurança marcou o material para revisão humana; mesmo assim, o restante do pipeline seguiu normalmente e gerou os quatro formatos de documento.

2.7 Áudio

```
renderers.sintetizador_de_voz:_sintetizar:126 - Gerando audio (TTS):  
1/2 blocos concluidos (50%) ... 2/2 blocos concluidos (100%)  
renderers.sintetizador_de_voz:export_mp3:182 - Audio (MP3) granular  
exportado com sucesso (2 blocos)
```

O que está acontecendo: o texto final (já em português, sem símbolo cru nenhum) foi dividido em 2 blocos e sintetizado em voz. Aqui vale lembrar de um bug que já corrigimos antes: o áudio só é aceito se TODOS os blocos forem gerados com sucesso, pra nunca ter problema com um arquivo cortado.

3. O primeiro B.O.: fórmula dentro do texto

O problema mais antigo, e o que motivou a reescrita mais profunda da camada matemática, foi este: o pipeline classificava cada BLOCO INTEIRO como um único tipo: ou parágrafo ou fórmula. Nunca os dois.

Material didático real não funciona assim. Um parágrafo comum costuma ter fórmula no meio da frase:

"A função associada ao exemplo é $f(x) = x^2 - 5x + 6$. As raízes calculadas por Bhaskara aparecem no gráfico..."

Classificar o bloco inteiro como um tipo só produzia dois erros opostos, e os dois já apareceram em produção:

- o bloco virava “parágrafo”: a fórmula nunca chegava ao especialista de visão e saía achatada, como texto cru (“ $ax^2 + bx + c = 0$ ” em vez da fórmula de verdade, sem estrutura nenhuma para o leitor de tela navegar);
- o bloco virava “fórmula”: o especialista de fórmula recebia a região inteira, inclusive a frase em português ao redor e essa frase era DESCARTADA. O aluno perdia parte do texto original.

Foi esse segundo caso, na prática, que rendeu o material de lógica em que um título de seção (“B = Operações entre conjuntos”) foi processado como se fosse uma fórmula a palavra virou produto de letras e a fala saía “b vezes o vezes p vezes e...”. Contaminação de região: texto que não é matemática entrando no nó matemático.

3.1 A correção: segmentar, não classificar

A solução ficou no módulo

`pipeline/matematica/matematica_inline.py`, que resolve o bloco em SEGMENTOS em vez de um tipo só:

```
[texto: "A função associada ao exemplo é "]  
[math: " $f(x) = x^2 - 5x + 6$ "]
```

[texto: ". As raízes calculadas por Bhaskara aparecem no gráfico..."]

Duas garantias foram embutidas nesse módulo, e as duas são testadas automaticamente:

- Invariante de preservação: a concatenação de todos os segmentos reproduz o texto original, caractere por caractere. Nenhuma frase pode desaparecer ao promover um trecho interno a fórmula.
- Detector conservador: só abre um segmento de fórmula quando há um sinal matemático FORTE no trecho tipo: igual (=), mais-ou-menos ($\pm$), raiz, delta, maior-ou-igual, menor-ou-igual, expoente, índice. Prosa comum nunca vira fórmula por engano; na pior hipótese, uma fórmula passa despercebida como texto.

Do lado de fora do parágrafo, quando a região inteira é candidata a fórmula (não está misturada em frase), quem decide é a heurística de

`core/classificador_de_regioes.py`, função `parece_formula()`. Ela também é conservadora: exige texto curto (fórmula de material didático não passa de ~220 caracteres), presença de símbolo forte, notação de função ou coeficiente colado, e baixa proporção de “palavras de prosa” no trecho. Foi calibrada com casos reais capturados em produção... Inclusive fórmulas embaralhadas pelo OCR, como “Vb2 $X = 2a = b + Aac$ ” (a própria fórmula de Bhaskara, desfigurada), que precisa continuar sendo reconhecida como fórmula mesmo fora de forma.

O efeito prático dessas duas correções juntas: hoje uma frase com fórmula embutida preserva tanto o texto quanto a fórmula, cada um pelo caminho certo, o texto sai como está, e a fórmula ganha a estrutura completa (MathML navegável, fala em português, LaTeX para braille etc) .

4. Outros problemas de semântica que apareceram no caminho

O “fórmula vira texto / texto vira fórmula” foi o primeiro e o mais estrutural, mas não foi o único. Registro aqui os principais, na ordem em que foram descobertos, a maioria a partir de material real gerado pelo próprio sistema, não de suposição. Tudo por tentativa e erro.

4.1 Letras soletradas de menos, ou de mais

A fala de uma fórmula tinha que decidir como pronunciar cada letra. Numa fase, só as letras ambíguas para o sintetizador de voz eram soletradas (o “x”, que sai como “xis”); depois o alfabeto inteiro passou a ser soletrado (“b” virava “bê”, “c” virava “cê”); a versão atual reverteu isso, cada letra fala como ela mesma porque soletrar tudo deixava a fala cansativa de ouvir sem muito ganho real. O ponto importante: essa política vive num único lugar do código (o vocabulário de fala), então mudar de ideia de novo é uma linha, não uma caçada por todo o sistema.

4.2 Unidades sendo lidas como variáveis

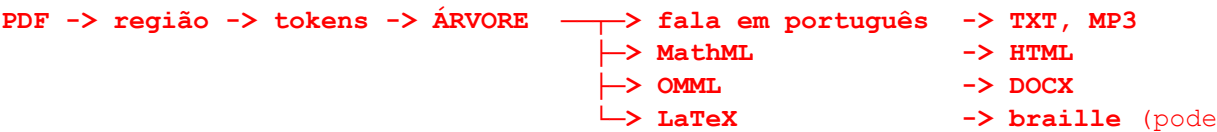
“5 m/s²” estava sendo interpretado como uma fração entre as variáveis “m” e “s ao quadrado” — a fala saía “fração, numerador m, denominador s ao quadrado” em vez de “cinco metros por segundo ao quadrado”. Criamos um tipo de nó próprio para quantidade-com-unidade, com uma tabela fechada de unidades do SI (metro, segundo, quilograma, newton...) e concordância de plural.

4.3 Sinais soltos: “+”, “-” e produtos colados (o “4ac”/“2a” de novo)

Em células de tabela sem símbolo delimitador nenhum, “+” e “-” apareciam crus na fala (“usando + e outra usando -”), e o conversor de números colava o algarismo na letra seguinte (“4ac” virava “quatroac”; “2a” virava “doisa”). A correção mais recente: “+” sempre vira “mais”; “-” passa por uma auditoria — hífen entre letras coladas dos dois lados é palavra composta (“guarda-chuva”) e fica; todo o resto é operador matemático e vira “menos”. Apenas o “quatroac” não foi resolvido até o presente momento.

5. Como o código está organizado hoje

A ideia central, depois de todos esses ajustes: cada fórmula é reconhecida UMA vez e vira uma árvore. A partir daí, cada formato de saída é apenas uma forma diferente de “ler” essa mesma árvore — nunca uma segunda interpretação por conta própria.



Essa regra evita exatamente o tipo de bug descrito acima: se cada formato reconstruísse a fórmula à sua maneira, o TXT podia dizer uma coisa e o HTML dizer outra, e a divergência só aparece quando alguém compara os dois de propósito.

5.1 Módulos e o que cada um faz

Módulo	O que faz
core/classificador_de_regioes.py	Decide o que cada região extraída do PDF é (texto, fórmula, tabela, imagem). Contém a heurística parece_formula(), calibrada com casos reais de OCR embaralhado.
pipeline/matematica/matematica_inline.py	Resolve um bloco de texto em segmentos de texto e fórmula, em vez de classificar o bloco inteiro de uma vez. É a

	correção do primeiro B.O. descrito na seção 3.
<code>pipeline/matematica/arvore_matematica.py</code>	Tokeniza e monta a árvore (AST) a partir do LaTeX ou do texto reconhecido. É a fonte única de verdade — todo o resto deriva daqui.
<code>pipeline/matematica/fala_matematica.py</code>	Gera a fala em português a partir da árvore, em dois modos: estrutural (para o TXT, anuncia “no numerador / no denominador”) e conciso (leitura corrida).
<code>pipeline/matematica/serializacao_matematica.py</code>	Gera LaTeX, MathML e OMML a partir da MESMA árvore — inclui o Invisible Times da multiplicação implícita e os ids de navegação do MathML.
<code>pipeline/matematica/cobertura_matematica.py</code>	Os validadores: comparam o

	que entrou com o que a árvore entendeu. Bloqueia publicação se sobrar símbolo cru, operador perdido, MathML inválido, texto de título incorporado à fórmula, entre outros.
pipeline/matematica/verbalizador_de_simbolos.py	Trata matemática solta em texto corrido e em células de tabela — inclui a auditoria de “+”/“-” e o reconhecimento de produtos colados (4ac, 2a) descritos na seção 4.5.
pipeline/matematica/problemas_matematicos.py	Assinatura estrutural da árvore, usada para decidir se duas fórmulas são realmente a mesma (seção 4.4) — nunca por aparência visual.

<code>core/agents/critico_visual.py</code>	Compara a descrição gerada pelo especialista contra a imagem original antes de aceitar; se reprovar, manda o especialista tentar de novo.
<code>pipeline/matematica/podador.py</code>	Remove ruído do documento já montado (regiões sem conteúdo real, duplicatas) antes da exportação final.
<code>renderers/renderizador_html.py, _txt.py, _docx.py</code>	Cada um lê a mesma árvore e produz seu formato — nenhum reprocessa a fórmula do zero.
<code>renderers/sintetizador_de_voz.py</code>	Gera o MP3 a partir do TXT já pronto. Só entrega o áudio se TODOS os blocos forem sintetizados com sucesso.

6. Pendência identificada neste log: módulo de metadados ausente

O aviso `Metadado ACESSILIA nao registrado (No module named 'core.services.acessilia_meta')` apareceu quatro vezes nesta execução, uma para cada região visual processada (as três fórmulas e a tabela).

O que esse módulo faz, quando presente: grava, para cada descrição gerada por IA, um registro de rastreabilidade, tipo o nível de confiança, modelo usado, data, e o status de revisão (rascunho, revisado, aprovado). É a ficha que permite auditar depois por que uma descrição específica foi aceita.

A boa notícia é que a ausência dele não interrompeu nada: o material foi processado e os artefatos foram gerados normalmente. O agente trata essa chamada como opcional (o padrão “fail-open” do projeto ; nenhum registro complementar pode derrubar a entrega do material).

Isso foi feito em outro momento, não foi necessário pra agora. É o esboço de outra ideia.

7. Resultado final desta execução

Apesar do aviso pendente da seção 6, a execução analisada terminou com sucesso: os quatro formatos de documento (DOCX, HTML, PDF, TXT) e o áudio MP3 foram gerados e entregues. As três fórmulas e a tabela foram descritas com confiança 0.98 pelo crítico visual, o podador removeu um único bloco de ruído sem descartar conteúdo real, e nenhuma duplicata falsa foi detectada, o que confirma, na prática, que as correções das seções 3 e 4 estão de fato valendo em material real, não só nos testes automatizados.

Fica registrado como aprendizado central deste processo: o padrão que mais vezes causou problema foi tratar um formato de saída (imagem, texto, layout) como se ele já contivesse o significado, em vez de reconhecer o significado uma vez e derivar cada formato dele. As correções descritas aqui, segmentação de texto e fórmula, unidades como tipo próprio, comparação por estrutura em vez de aparência, um único módulo para os sinais residuais de +/- — são, todas, a mesma ideia aplicada em pontos diferentes do pipeline.